
MERN Student Management System

Complete One-Paste Coding Prompt

Build a complete **Student Management System** using the MERN stack.
Suitable for a beginner-to-intermediate full-stack student project with clean, professional architecture.

1. Technology Stack

Frontend

- React
- Vite
- Tailwind CSS
- React Router DOM
- Axios
- Lucide React

Backend

- Node.js
- Express.js
- MongoDB
- Mongoose
- JWT
- bcryptjs
- cookie-parser
- CORS
- dotenv

Authentication

Use:

- JWT authentication
- **httpOnly** cookies
- Secure cookies in production
- **SameSite** configuration
- No JWT in localStorage
- No JWT in sessionStorage
- No Authorization Bearer token from React

The browser must automatically send the authentication cookie with API requests.

2. Project Name

Create:

```
student-management-system
```

Structure:

```
student-management-system/  
├──  
│   ├── backend/  
│   └── frontend/
```

3. Database Collections

Use exactly these five main MongoDB collections:

1. Users
2. Students
3. Courses
4. Enrollments
5. Attendance

Relationships:

```
User  
  ├── Authentication  
  
Student  
  ├── Enrollments  
  │   ├── Course  
  │   └──  
  ├── Attendance  
  │   └── Course
```

Use MongoDB ObjectId references and Mongoose `populate()` where appropriate.

4. Backend Folder Structure

Create:

```
backend/
├──
├── config/
│   └── db.js
├──
├── controllers/
│   ├── authController.js
│   ├── studentController.js
│   ├── courseController.js
│   ├── enrollmentController.js
│   └── attendanceController.js
├──
├── middleware/
│   ├── authMiddleware.js
│   └── errorMiddleware.js
├──
├── models/
│   ├── User.js
│   ├── Student.js
│   ├── Course.js
│   ├── Enrollment.js
│   └── Attendance.js
├──
├── routes/
│   ├── authRoutes.js
│   ├── studentRoutes.js
│   ├── courseRoutes.js
│   ├── enrollmentRoutes.js
│   └── attendanceRoutes.js
├──
├── utils/
│   └── generateToken.js
├──
├── .env
├── package.json
└── server.js
```

5. Backend Installation

Run:

```
mkdir student-management-system
cd student-management-system

mkdir backend
cd backend

npm init -y

npm install express mongoose bcryptjs jsonwebtoken dotenv cors cookie-parser

npm install -D nodemon
```

Configure `package.json`:

```
{
  "name": "student-management-backend",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  }
}
```

6. Backend Environment Variables

Create:

```
backend/.env
```

Add:

```
PORT=5000

MONGO_URI=mongodb://127.0.0.1:27017/student_management

JWT_SECRET=change_this_to_a_long_random_secret_key

JWT_EXPIRES_IN=1d

CLIENT_URL=http://localhost:5173

NODE_ENV=development
```

Do not commit `.env` to GitHub.

7. MongoDB Connection

File:

```
backend/config/db.js
```

Code:

```
import mongoose from "mongoose";

const connectDB = async () => {
  try {
    const connection = await mongoose.connect(
      process.env.MONGO_URI
    );

    console.log(
      `MongoDB connected: ${connection.connection.host}`
    );
  } catch (error) {
    console.error(
      "MongoDB connection failed:",
      error.message
    );

    process.exit(1);
  }
};

export default connectDB;
```

8. User Model

File:

```
backend/models/User.js
```

Code:

```

import mongoose from "mongoose";
import bcrypt from "bcryptjs";

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true
    },

    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true
    },

    password: {
      type: String,
      required: true,
      minlength: 6
    },

    role: {
      type: String,
      enum: ["admin", "staff"],
      default: "staff"
    }
  },
  {
    timestamps: true
  }
);

userSchema.pre("save", async function (next) {
  if (!this.isModified("password")) {
    return next();
  }

  const salt = await bcrypt.genSalt(10);

  this.password = await bcrypt.hash(
    this.password,
    salt
  );

  next();
});

userSchema.methods.comparePassword =
  async function (password) {
    return bcrypt.compare(
      password,
      this.password
    );
  };

const User = mongoose.model(
  "User",
  userSchema
);

export default User;

```

9. Student Model

File:

```
backend/models/Student.js
```

Code:

```
import mongoose from "mongoose";

const studentSchema = new mongoose.Schema(
  {
    studentId: {
      type: String,
      required: true,
      unique: true,
      trim: true
    },
    firstName: {
      type: String,
      required: true,
      trim: true
    },
    lastName: {
      type: String,
      required: true,
      trim: true
    },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true
    },
    phone: {
      type: String,
      required: true,
      trim: true
    },
    gender: {
      type: String,
      enum: ["Male", "Female", "Other"],
      required: true
    },
    dateOfBirth: {
      type: Date,
      required: true
    },
    address: {
      type: String,
      trim: true
    },
    guardianName: {
      type: String,
```

```
      trim: true
    },

    guardianPhone: {
      type: String,
      trim: true
    },

    status: {
      type: String,
      enum: ["Active", "Inactive"],
      default: "Active"
    }
  },
  {
    timestamps: true
  }
);

const Student = mongoose.model(
  "Student",
  studentSchema
);

export default Student;
```

10. Course Model

File:

```
backend/models/Course.js
```

Code:

```
import mongoose from "mongoose";

const courseSchema = new mongoose.Schema(
  {
    courseCode: {
      type: String,
      required: true,
      unique: true,
      uppercase: true,
      trim: true
    },
    courseName: {
      type: String,
      required: true,
      trim: true
    },
    description: {
      type: String,
      trim: true
    },
    duration: {
      type: String,
      required: true
    },
    fee: {
      type: Number,
      required: true,
      min: 0
    },
    status: {
      type: String,
      enum: ["Active", "Inactive"],
      default: "Active"
    }
  },
  {
    timestamps: true
  }
);

const Course = mongoose.model(
  "Course",
  courseSchema
);

export default Course;
```

11. Enrollment Model

File:

```
backend/models/Enrollment.js
```

Code:

```

import mongoose from "mongoose";

const enrollmentSchema = new mongoose.Schema(
  {
    student: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Student",
      required: true
    },

    course: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Course",
      required: true
    },

    enrollmentDate: {
      type: Date,
      default: Date.now
    },

    batch: {
      type: String,
      required: true,
      trim: true
    },

    status: {
      type: String,
      enum: [
        "Active",
        "Completed",
        "Dropped"
      ],
      default: "Active"
    }
  },
  {
    timestamps: true
  }
);

enrollmentSchema.index(
  {
    student: 1,
    course: 1,
    batch: 1
  },
  {
    unique: true
  }
);

const Enrollment = mongoose.model(
  "Enrollment",

```

```

  enrollmentSchema
);

export default Enrollment;

```

12. Attendance Model

File:

backend/models/Attendance.js

Code:

```
import mongoose from "mongoose";

const attendanceSchema = new mongoose.Schema(
  {
    student: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Student",
      required: true
    },
    course: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Course",
      required: true
    },
    date: {
      type: Date,
      required: true
    },
    status: {
      type: String,
      enum: [
        "Present",
        "Absent",
        "Late"
      ],
      required: true
    },
    remarks: {
      type: String,
      trim: true
    }
  },
  {
    timestamps: true
  }
);

attendanceSchema.index(
  {
    student: 1,
    course: 1,
    date: 1
  },
  {
    unique: true
  }
);

const Attendance = mongoose.model(
  "Attendance",
  attendanceSchema
);

export default Attendance;
```

13. JWT Utility

File:

```
backend/utils/generateToken.js
```

Code:

```
import jwt from "jsonwebtoken";

const generateToken = (userId) => {
  return jwt.sign(
    {
      userId
    },
    process.env.JWT_SECRET,
    {
      expiresIn:
        process.env.JWT_EXPIRES_IN || "1d"
    }
  );
};

export default generateToken;
```

14. Authentication Middleware

File:

```
backend/middleware/authMiddleware.js
```

Code:

```
import jwt from "jsonwebtoken";
import User from "../models/User.js";

const protect = async (
  req,
  res,
  next
) => {
  try {
    const token = req.cookies.token;

    if (!token) {
      return res.status(401).json({
        message: "Authentication required"
      });
    }

    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET
    );

    const user = await User.findById(
      decoded.userId
    ).select("-password");

    if (!user) {
      return res.status(401).json({
        message: "User not found"
      });
    }

    req.user = user;

    next();
  } catch (error) {
    return res.status(401).json({
      message:
        "Invalid or expired token"
    });
  }
};

export default protect;
```

15. Error Middleware

File:

```
backend/middleware/errorMiddleware.js
```

Code:

```
const errorMiddleware = (
  err,
  req,
  res,
  next
) => {
  console.error(err);

  if (err.code === 11000) {
    const field =
      Object.keys(
        err.keyPattern
      )[0];

    return res.status(400).json({
      message:
        `${field} already exists`
    });
  }

  if (
    err.name === "ValidationError"
  ) {
    const errors =
      Object.values(
        err.errors
      ).map(
        (error) =>
          error.message
      );

    return res.status(400).json({
      message:
        errors.join(", ")
    });
  }

  if (err.name === "CastError") {
    return res.status(400).json({
      message: "Invalid ID"
    });
  }

  res.status(
    err.statusCode || 500
  ).json({
    message:
      err.message ||
      "Server error"
  });
};

export default errorMiddleware;
```

16. Authentication Controller

File:

```
backend/controllers/authController.js
```

Code:

```

import User from "../models/User.js";
import generateToken from "../utils/generateToken.js";

const getCookieOptions = () => ({
  httpOnly: true,

  secure:
    process.env.NODE_ENV ===
    "production",

  sameSite:
    process.env.NODE_ENV ===
    "production"
      ? "none"
      : "lax",

  maxAge:
    24 * 60 * 60 * 1000,

  path: "/"
});

export const register =
  async (req, res) => {
    const {
      name,
      email,
      password,
      role
    } = req.body;

    if (
      !name ||
      !email ||
      !password
    ) {
      return res.status(400).json({
        message:
          "Name, email and password are required"
      });
    }

    const existingUser =
      await User.findOne({
        email
      });

    if (existingUser) {
      return res.status(400).json({
        message:
          "Email already registered"
      });
    }

    const user =

```

```

    await User.create({
      name,
      email,
      password,
      role: role || "staff"
    });

    const token =
      generateToken(
        user._id
      );

    res.cookie(
      "token",
      token,
      getCookieOptions()
    );

    res.status(201).json({
      message:
        "Registration successful",

      user: {
        id: user._id,
        name: user.name,
        email: user.email,
        role: user.role
      }
    });
  };
};

export const login =
  async (req, res) => {
    const {
      email,
      password
    } = req.body;

    if (
      !email ||
      !password
    ) {
      return res.status(400).json({
        message:
          "Email and password are required"
      });
    }

    const user =
      await User.findOne({
        email
      });

    if (!user) {
      return res.status(401).json({

```

```

        message:
          "Invalid email or password"
      });
    }

    const passwordMatch =
      await user.comparePassword(
        password
      );

    if (!passwordMatch) {
      return res.status(401).json({
        message:
          "Invalid email or password"
      });
    }

    const token =
      generateToken(
        user._id
      );

    res.cookie(
      "token",
      token,
      getCookieOptions()
    );

    res.json({
      message:
        "Login successful",

      user: {
        id: user._id,
        name: user.name,
        email: user.email,
        role: user.role
      }
    });
  };

export const logout =
  async (req, res) => {
    res.clearCookie(
      "token",
      {
        httpOnly: true,

        secure:
          process.env.NODE_ENV ===
            "production",

        sameSite:
          process.env.NODE_ENV ===
            "production"
      }
    );
  };

```

```
      ? "none"
      : "lax",

      path: "/"
    }
  );

  res.json({
    message:
      "Logout successful"
  });
};

export const getMe =
  async (req, res) => {
    res.json({
      user: req.user
    });
  };
};
```

17. Student Controller

File:

```
backend/controllers/studentController.js
```

Code:

```
import Student from "../models/Student.js";

export const getStudents =
  async (req, res) => {
    const {
      search = "",
      status,
      page = 1,
      limit = 10
    } = req.query;

    const query = {};

    if (search) {
      query.$or = [
        {
          firstName: {
            $regex: search,
            $options: "i"
          }
        },
        {
          lastName: {
            $regex: search,
            $options: "i"
          }
        },
        {
          studentId: {
            $regex: search,
            $options: "i"
          }
        },
        {
          email: {
            $regex: search,
            $options: "i"
          }
        }
      ];
    }

    if (status) {
      query.status =
        status;
    }

    const pageNumber =
      Number(page);

    const limitNumber =
      Number(limit);

    const skip =
      (pageNumber - 1) *
```

```

        limitNumber;

const [
  students,
  total
] = await Promise.all([
  Student.find(query)
    .sort({
      createdAt: -1
    })
    .skip(skip)
    .limit(limitNumber),

  Student.countDocuments(
    query
  )
]);

res.json({
  students,

  pagination: {
    page: pageNumber,
    limit: limitNumber,
    total,
    totalPages:
      Math.ceil(
        total /
        limitNumber
      )
  }
});
};

export const getStudent =
  async (req, res) => {
    const student =
      await Student.findById(
        req.params.id
      );

    if (!student) {
      return res.status(404).json({
        message:
          "Student not found"
      });
    }

    res.json({
      student
    });
  };

export const createStudent =
  async (req, res) => {

```

```

    const student =
      await Student.create(
        req.body
      );

    res.status(201).json({
      message:
        "Student created successfully",
      student
    });
  });
};

export const updateStudent =
  async (req, res) => {
    const student =
      await Student.findByIdAndUpdate(
        req.params.id,
        req.body,
        {
          new: true,
          runValidators: true
        }
      );

    if (!student) {
      return res.status(404).json({
        message:
          "Student not found"
      });
    }

    res.json({
      message:
        "Student updated successfully",
      student
    });
  });

export const deleteStudent =
  async (req, res) => {
    const student =
      await Student.findByIdAndDelete(
        req.params.id
      );

    if (!student) {
      return res.status(404).json({
        message:
          "Student not found"
      });
    }

    res.json({
      message:
        "Student deleted successfully"
    });
  });
};

```

18. Course Controller

File:

```
backend/controllers/courseController.js
```

Code:

```
import Course from "../models/Course.js";

export const getCourses =
  async (req, res) => {
    const {
      search = "",
      status
    } = req.query;

    const query = {};

    if (search) {
      query.$or = [
        {
          courseCode: {
            $regex: search,
            $options: "i"
          }
        },
        {
          courseName: {
            $regex: search,
            $options: "i"
          }
        }
      ];
    }

    if (status) {
      query.status =
        status;
    }

    const courses =
      await Course.find(
        query
      ).sort({
        createdAt: -1
      });

    res.json({
      courses
    });
  };

export const getCourse =
  async (req, res) => {
    const course =
      await Course.findById(
        req.params.id
      );

    if (!course) {
      return res.status(404).json({
        message:

```

```

        "Course not found"
    });
}

res.json({
  course
});
};

export const createCourse =
  async (req, res) => {
    const course =
      await Course.create(
        req.body
      );

    res.status(201).json({
      message:
        "Course created successfully",
      course
    });
  };

export const updateCourse =
  async (req, res) => {
    const course =
      await Course.findByIdAndUpdate(
        req.params.id,
        req.body,
        {
          new: true,
          runValidators: true
        }
      );

    if (!course) {
      return res.status(404).json({
        message:
          "Course not found"
      });
    }

    res.json({
      message:
        "Course updated successfully",
      course
    });
  };

export const deleteCourse =
  async (req, res) => {
    const course =
      await Course.findByIdAndDelete(
        req.params.id
      );

```

```

    if (!course) {
      return res.status(404).json({
        message:
          "Course not found"
      });
    }

    res.json({
      message:
        "Course deleted successfully"
    });
  };
};

```

19. Enrollment Controller

File:

```
backend/controllers/enrollmentController.js
```

Code:

```
import Enrollment from "../models/Enrollment.js";

export const getEnrollments =
  async (req, res) => {
    const {
      course,
      student,
      status
    } = req.query;

    const query = {};

    if (course) {
      query.course =
        course;
    }

    if (student) {
      query.student =
        student;
    }

    if (status) {
      query.status =
        status;
    }

    const enrollments =
      await Enrollment.find(
        query
      )
        .populate(
          "student",
          "studentId firstName lastName email"
        )
        .populate(
          "course",
          "courseCode courseName fee"
        )
        .sort({
          enrollmentDate:
            -1
        });

    res.json({
      enrollments
    });
  };

export const getEnrollment =
  async (req, res) => {
    const enrollment =
      await Enrollment.findById(
        req.params.id
      )
  }
```

```

        .populate("student")
        .populate("course");

    if (!enrollment) {
        return res.status(404).json({
            message:
                "Enrollment not found"
        });
    }

    res.json({
        enrollment
    });
};

export const createEnrollment =
    async (req, res) => {
        const enrollment =
            await Enrollment.create(
                req.body
            );

        const populatedEnrollment =
            await Enrollment.findById(
                enrollment._id
            )
                .populate("student")
                .populate("course");

        res.status(201).json({
            message:
                "Enrollment created successfully",

            enrollment:
                populatedEnrollment
        });
    };

export const updateEnrollment =
    async (req, res) => {
        const enrollment =
            await Enrollment.findByIdAndUpdate(
                req.params.id,
                req.body,
                {
                    new: true,
                    runValidators: true
                }
            )
                .populate("student")
                .populate("course");

        if (!enrollment) {
            return res.status(404).json({
                message:

```

```
        "Enrollment not found"
    });
}

res.json({
  message:
    "Enrollment updated successfully",

  enrollment
});
};

export const deleteEnrollment =
  async (req, res) => {
    const enrollment =
      await Enrollment.findByIdAndDelete(
        req.params.id
      );

    if (!enrollment) {
      return res.status(404).json({
        message:
          "Enrollment not found"
      });
    }

    res.json({
      message:
        "Enrollment deleted successfully"
    });
  };
};
```

20. Attendance Controller

File:

```
backend/controllers/attendanceController.js
```

Code:

```
import Attendance from "../models/Attendance.js";
```

```
export const getAttendance =
```

```
  async (req, res) => {
```

```
    const {
```

```
      course,
```

```
      student,
```

```
      date,
```

```
      status
```

```
    } = req.query;
```

```
    const query = {};
```

```
    if (course) {
```

```
      query.course =
```

```
        course;
```

```
    }
```

```
    if (student) {
```

```
      query.student =
```

```
        student;
```

```
    }
```

```
    if (status) {
```

```
      query.status =
```

```
        status;
```

```
    }
```

```
    if (date) {
```

```
      const start =
```

```
        new Date(date);
```

```
      start.setHours(
```

```
        0,
```

```
        0,
```

```
        0,
```

```
        0
```

```
      );
```

```
      const end =
```

```
        new Date(start);
```

```
      end.setDate(
```

```
        end.getDate() + 1
```

```
      );
```

```
      query.date = {
```

```
        $gte: start,
```

```
        $lt: end
```

```
      };
```

```
    }
```

```
    const attendance =
```

```
      await Attendance.find(
```

```
        query
```

```

    )
    .populate(
      "student",
      "studentId firstName lastName"
    )
    .populate(
      "course",
      "courseCode courseName"
    )
    .sort({
      date: -1
    });

    res.json({
      attendance
    });
  };

export const getAttendanceRecord =
  async (req, res) => {
    const record =
      await Attendance.findById(
        req.params.id
      )
      .populate("student")
      .populate("course");

    if (!record) {
      return res.status(404).json({
        message:
          "Attendance record not found"
      });
    }

    res.json({
      attendance:
        record
    });
  };

export const createAttendance =
  async (req, res) => {
    const {
      student,
      course,
      date,
      status,
      remarks
    } = req.body;

    const attendanceDate =
      new Date(date);

    const start =
      new Date(

```

```

        attendanceDate
    );

    start.setHours(
        0,
        0,
        0,
        0
    );

    const end =
        new Date(start);

    end.setDate(
        end.getDate() + 1
    );

    const existing =
        await Attendance.findOne({
            student,
            course,
            date: {
                $gte: start,
                $lt: end
            }
        });

    if (existing) {
        return res.status(400).json({
            message:
                "Attendance already marked for this student"
        });
    }

    const record =
        await Attendance.create({
            student,
            course,
            date: start,
            status,
            remarks
        });

    const populated =
        await Attendance.findById(
            record._id
        )
            .populate("student")
            .populate("course");

    res.status(201).json({
        message:
            "Attendance marked successfully",

        attendance:

```

```

        populated
    });
};

export const updateAttendance =
  async (req, res) => {
    const record =
      await Attendance.findByIdAndUpdate(
        req.params.id,
        req.body,
        {
          new: true,
          runValidators: true
        }
      )
      .populate("student")
      .populate("course");

    if (!record) {
      return res.status(404).json({
        message:
          "Attendance record not found"
      });
    }

    res.json({
      message:
        "Attendance updated successfully",

      attendance:
        record
    });
  };

export const deleteAttendance =
  async (req, res) => {
    const record =
      await Attendance.findByIdAndDelete(
        req.params.id
      );

    if (!record) {
      return res.status(404).json({
        message:
          "Attendance record not found"
      });
    }

    res.json({
      message:
        "Attendance deleted successfully"
    });
  };
};

```

21. Auth Routes

File:

```
backend/routes/authRoutes.js
```

Code:

```
import express from "express";

import {
  register,
  login,
  logout,
  getMe
} from "../controllers/authController.js";

import protect from "../middleware/authMiddleware.js";

const router =
  express.Router();

router.post(
  "/register",
  register
);

router.post(
  "/login",
  login
);

router.post(
  "/logout",
  logout
);

router.get(
  "/me",
  protect,
  getMe
);

export default router;
```

22. Student Routes

File:

```
backend/routes/studentRoutes.js
```

Code:

```
import express from "express";

import {
  getStudents,
  getStudent,
  createStudent,
  updateStudent,
  deleteStudent
} from "../controllers/studentController.js";

import protect from "../middleware/authMiddleware.js";

const router =
  express.Router();

router.use(protect);

router.get(
  "/",
  getStudents
);

router.post(
  "/",
  createStudent
);

router.get(
  "/:id",
  getStudent
);

router.put(
  "/:id",
  updateStudent
);

router.delete(
  "/:id",
  deleteStudent
);

export default router;
```

23. Course Routes

File:

```
backend/routes/courseRoutes.js
```

Code:

```
import express from "express";

import {
  getCourses,
  getCourse,
  createCourse,
  updateCourse,
  deleteCourse
} from "../controllers/courseController.js";

import protect from "../middleware/authMiddleware.js";

const router =
  express.Router();

router.use(protect);

router.get(
  "/",
  getCourses
);

router.post(
  "/",
  createCourse
);

router.get(
  "/:id",
  getCourse
);

router.put(
  "/:id",
  updateCourse
);

router.delete(
  "/:id",
  deleteCourse
);

export default router;
```

24. Enrollment Routes

File:

```
backend/routes/enrollmentRoutes.js
```

Code:

```
import express from "express";

import {
  getEnrollments,
  getEnrollment,
  createEnrollment,
  updateEnrollment,
  deleteEnrollment
} from "../controllers/enrollmentController.js";

import protect from "../middleware/authMiddleware.js";

const router =
  express.Router();

router.use(protect);

router.get(
  "/",
  getEnrollments
);

router.post(
  "/",
  createEnrollment
);

router.get(
  "/:id",
  getEnrollment
);

router.put(
  "/:id",
  updateEnrollment
);

router.delete(
  "/:id",
  deleteEnrollment
);

export default router;
```

25. Attendance Routes

File:

```
backend/routes/attendanceRoutes.js
```

Code:

```
import express from "express";

import {
  getAttendance,
  getAttendanceRecord,
  createAttendance,
  updateAttendance,
  deleteAttendance
} from "../controllers/attendanceController.js";

import protect from "../middleware/authMiddleware.js";

const router =
  express.Router();

router.use(protect);

router.get(
  "/",
  getAttendance
);

router.post(
  "/",
  createAttendance
);

router.get(
  "/:id",
  getAttendanceRecord
);

router.put(
  "/:id",
  updateAttendance
);

router.delete(
  "/:id",
  deleteAttendance
);

export default router;
```

26. Backend Server

File:

```
backend/server.js
```

Code:

```

import express from "express";
import cors from "cors";
import dotenv from "dotenv";
import cookieParser from "cookie-parser";

import connectDB from "../config/db.js";

import authRoutes from "../routes/authRoutes.js";
import studentRoutes from "../routes/studentRoutes.js";
import courseRoutes from "../routes/courseRoutes.js";
import enrollmentRoutes from "../routes/enrollmentRoutes.js";
import attendanceRoutes from "../routes/attendanceRoutes.js";

import errorMiddleware from "../middleware/errorMiddleware.js";

dotenv.config();

connectDB();

const app =
  express();

app.use(
  cors({
    origin:
      process.env.CLIENT_URL,
    credentials: true
  })
);

app.use(
  express.json()
);

app.use(
  express.urlencoded({
    extended: true
  })
);

app.use(
  cookieParser()
);

app.get(
  "/",
  (req, res) => {
    res.json({
      message:
        "Student Management API is running"
    });
  }
);

app.use(

```

```
    "/api/auth",
    authRoutes
  );

  app.use(
    "/api/students",
    studentRoutes
  );

  app.use(
    "/api/courses",
    courseRoutes
  );

  app.use(
    "/api/enrollments",
    enrollmentRoutes
  );

  app.use(
    "/api/attendance",
    attendanceRoutes
  );

  app.use(
    errorMiddleware
  );

  const PORT =
    process.env.PORT ||
    5000;

  app.listen(
    PORT,
    () => {
      console.log(
        `Server running on http://localhost:${PORT}`
      );
    }
  );
```

27. Start Backend

Run:

```
cd backend
npm run dev
```

Expected:

```
MongoDB connected
Server running on http://localhost:5000
```

28. Frontend Setup

From the project root:

```
cd ..  
  
npm create vite@latest frontend -- --template react  
  
cd frontend  
  
npm install  
  
npm install axios react-router-dom lucide-react
```

Install and configure Tailwind CSS using the current Tailwind/Vite setup.

29. Frontend Environment

Create:

```
frontend/.env
```

Add:

```
VITE_API_URL=http://localhost:5000/api
```

30. Frontend Folder Structure

Create:

```
frontend/src/
├──
├── components/
│   ├── Navbar.jsx
│   ├── Sidebar.jsx
│   ├── ProtectedRoute.jsx
│   ├── Loading.jsx
│   └── ConfirmModal.jsx
├──
├── context/
│   └── AuthContext.jsx
├──
├── pages/
│   ├── Login.jsx
│   ├── Register.jsx
│   ├── Dashboard.jsx
│   ├── Students.jsx
│   ├── AddStudent.jsx
│   ├── EditStudent.jsx
│   ├── Courses.jsx
│   ├── AddCourse.jsx
│   ├── EditCourse.jsx
│   ├── Enrollments.jsx
│   ├── AddEnrollment.jsx
│   ├── Attendance.jsx
│   ├── MarkAttendance.jsx
│   └── Profile.jsx
├──
├── services/
│   └── api.js
├──
├── App.jsx
├── main.jsx
└── index.css
```

31. Axios Service

File:

```
frontend/src/services/api.js
```

Code:

```
import axios from "axios";

const api =
  axios.create({
    baseURL:
      import.meta.env
        .VITE_API_URL,

    withCredentials: true
  });

export default api;
```

IMPORTANT:

Do not do:

```
localStorage.setItem(
  "token",
  token
);
```

Do not do:

```
Authorization: `Bearer ${token}`
```

Authentication is handled through the **httpOnly** cookie.

32. Auth Context

File:

```
frontend/src/context/AuthContext.jsx
```

Code:

```

import {
  createContext,
  useContext,
  useEffect,
  useState
} from "react";

import api from "../services/api";

const AuthContext =
  createContext();

export const AuthProvider = ({
  children
}) => {
  const [
    user,
    setUser
  ] = useState(null);

  const [
    loading,
    setLoading
  ] = useState(true);

  const checkAuth =
    async () => {
      try {
        const response =
          await api.get(
            "/auth/me"
          );

        setUser(
          response.data.user
        );
      } catch {
        setUser(null);
      } finally {
        setLoading(false);
      }
    };

  useEffect(() => {
    checkAuth();
  }, []);

  const login =
    async (
      email,
      password
    ) => {
      const response =
        await api.post(
          "/auth/login",

```

```

        {
            email,
            password
        }
    );

    setUser(
        response.data.user
    );

    return response.data;
};

```

```

const register =
    async (
        name,
        email,
        password
    ) => {
        const response =
            await api.post(
                "/auth/register",
                {
                    name,
                    email,
                    password
                }
            );

        setUser(
            response.data.user
        );

        return response.data;
    };

```

```

const logout =
    async () => {
        try {
            await api.post(
                "/auth/logout"
            );
        } finally {
            setUser(null);
        }
    };

```

```

return (
    <AuthContext.Provider
        value={{
            user,
            loading,
            login,
            register,
            logout

```

```

        }}
    >
        {children}
    </AuthContext.Provider>
);
};

```

```

export const useAuth =
    () =>
        useContext(
            AuthContext
        );

```

33. Protected Route

File:

```
frontend/src/components/ProtectedRoute.jsx
```

Code:

```
import {
  Navigate,
  Outlet
} from "react-router-dom";

import {
  useAuth
} from "../context/AuthContext";

const ProtectedRoute =
  () => {
    const {
      user,
      loading
    } = useAuth();

    if (loading) {
      return (
        <div className="flex min-h-screen items-center justify-center">
          Loading...
        </div>
      );
    }

    if (!user) {
      return (
        <Navigate
          to="/login"
          replace
        />
      );
    }

    return <Outlet />;
  };

export default ProtectedRoute;
```

34. Login Page

File:

```
frontend/src/pages/Login.jsx
```

Code:

```

import {
  useState
} from "react";

import {
  Link,
  useNavigate
} from "react-router-dom";

import {
  useAuth
} from "../context/AuthContext";

const Login =
() => {
  const navigate =
    useNavigate();

  const {
    login
  } = useAuth();

  const [
    form,
    setForm
  ] = useState({
    email: "",
    password: ""
  });

  const [
    error,
    setError
  ] = useState("");

  const [
    loading,
    setLoading
  ] = useState(false);

  const handleChange =
    (e) => {
      setForm({
        ...form,
        [e.target.name]:
          e.target.value
      });
    };

  const handleSubmit =
    async (e) => {
      e.preventDefault();

      setError("");
      setLoading(true);
    };

```

```

    try {
      await login(
        form.email,
        form.password
      );

      navigate(
        "/dashboard"
      );
    } catch (error) {
      setError(
        error.response
          ?.data?.message ||
        "Login failed"
      );
    } finally {
      setLoading(false);
    }
  };
};

return (
  <div className="flex min-h-screen items-center justify-center bg-gray-100 px-4">
    <div className="w-full max-w-md rounded-xl bg-white p-8 shadow">

      <h1 className="mb-2 text-2xl font-bold">
        Student Management System
      </h1>

      <p className="mb-6 text-gray-500">
        Login to your account
      </p>

      {error && (
        <div className="mb-4 rounded-lg bg-red-100 p-3 text-red-700">
          {error}
        </div>
      )}

      <form
        onSubmit={
          handleSubmit
        }
        className="space-y-4"
      >
        <input
          type="email"
          name="email"
          placeholder="Email"
          value={
            form.email
          }
          onChange={
            handleChange
          }
        >

```

```

        className="w-full rounded-lg border p-3"
        required
      />

      <input
        type="password"
        name="password"
        placeholder="Password"
        value={
          form.password
        }
        onChange={
          handleChange
        }
        className="w-full rounded-lg border p-3"
        required
      />

      <button
        disabled={
          loading
        }
        className="w-full rounded-lg bg-blue-600 p-3 font-medium text-white hover:bg-blue-700 disabled:opacity-50"
      >
        {loading
          ? "Logging in..."
          : "Login"}
      </button>
    </form>

    <p className="mt-6 text-center text-sm">
      Don't have an account?{" "}
      <Link
        to="/register"
        className="text-blue-600"
      >
        Register
      </Link>
    </p>
  </div>
</div>
);
};

export default Login;

```

35. Register Page

File:

```
frontend/src/pages/Register.jsx
```

Code:

```

import {
  useState
} from "react";

import {
  Link,
  useNavigate
} from "react-router-dom";

import {
  useAuth
} from "../context/AuthContext";

const Register =
() => {
  const navigate =
    useNavigate();

  const {
    register
  } = useAuth();

  const [
    form,
    setForm
  ] = useState({
    name: "",
    email: "",
    password: ""
  });

  const [
    error,
    setError
  ] = useState("");

  const handleChange =
    (e) => {
      setForm({
        ...form,
        [e.target.name]:
          e.target.value
      });
    };

  const handleSubmit =
    async (e) => {
      e.preventDefault();

      try {
        await register(
          form.name,
          form.email,
          form.password
        );
      }
    };

```

```

        navigate(
            "/dashboard"
        );
    } catch (error) {
        setError(
            error.response
                ?.data?.message ||
            "Registration failed"
        );
    }
};

return (
    <div className="flex min-h-screen items-center justify-center bg-gray-100 px-4">
        <div className="w-full max-w-md rounded-xl bg-white p-8 shadow">

            <h1 className="mb-6 text-2xl font-bold">
                Create Account
            </h1>

            {error && (
                <div className="mb-4 rounded bg-red-100 p-3 text-red-700">
                    {error}
                </div>
            )}

            <form
                onSubmit={
                    handleSubmit
                }
                className="space-y-4"
            >
                <input
                    name="name"
                    placeholder="Full Name"
                    value={
                        form.name
                    }
                    onChange={
                        handleChange
                    }
                    className="w-full rounded-lg border p-3"
                    required
                />

                <input
                    type="email"
                    name="email"
                    placeholder="Email"
                    value={
                        form.email
                    }
                    onChange={
                        handleChange

```

```

        }
        className="w-full rounded-lg border p-3"
        required
      />

      <input
        type="password"
        name="password"
        placeholder="Password"
        value={
          form.password
        }
        onChange={
          handleChange
        }
        className="w-full rounded-lg border p-3"
        minLength="6"
        required
      />

      <button className="w-full rounded-lg bg-blue-600 p-3 text-white">
        Create Account
      </button>
    </form>

    <p className="mt-6 text-center">
      Already have an account?{" "}
      <Link
        to="/login"
        className="text-blue-600"
      >
        Login
      </Link>
    </p>
  </div>
</div>
);
};

export default Register;

```

36. Navbar

File:

```
frontend/src/components/Navbar.jsx
```

Code:

```
import {
  useAuth
} from "../context/AuthContext";

const Navbar =
() => {
  const {
    user,
    logout
  } = useAuth();

  return (
    <header className="flex h-16 items-center justify-between border-b bg-white px-6">

      <h1 className="font-bold">
        Student Management
      </h1>

      <div className="flex items-center gap-4">

        <span className="text-sm text-gray-600">
          {user?.name}
        </span>

        <button
          onClick={logout}
          className="rounded-lg bg-red-500 px-4 py-2 text-sm text-white"
        >
          Logout
        </button>

      </div>

    </header>
  );
};

export default Navbar;
```

37. Sidebar

File:

```
frontend/src/components/Sidebar.jsx
```

Code:

```

import {
  NavLink
} from "react-router-dom";

const links = [
  {
    name: "Dashboard",
    path: "/dashboard"
  },
  {
    name: "Students",
    path: "/students"
  },
  {
    name: "Courses",
    path: "/courses"
  },
  {
    name: "Enrollments",
    path: "/enrollments"
  },
  {
    name: "Attendance",
    path: "/attendance"
  },
  {
    name: "Profile",
    path: "/profile"
  }
];

const Sidebar =
() => {
  return (
    <aside className="hidden w-64 border-r bg-white p-4 md:block">

      <nav className="space-y-2">

        {links.map(
          (link) => (
            <NavLink
              key={
                link.path
              }
              to={
                link.path
              }
              className={({
                isActive
              }) =>
                `block rounded-lg px-4 py-3 ${
                  isActive
                    ? "bg-blue-600 text-white"
                    : "text-gray-700 hover:bg-gray-100"
                }`
            )
          )
        }
      </nav>
    </aside>
  )
}

```

```
        }
      >
      {
        link.name
      }
    </NavLink>
  )
}
}

</nav>

</aside>
);
};

export default Sidebar;
```

38. Dashboard

File:

```
frontend/src/pages/Dashboard.jsx
```

Code:

```

import {
  useEffect,
  useState
} from "react";

import api from "../services/api";

const Dashboard =
() => {
  const [
    stats,
    setStats
  ] = useState({
    students: 0,
    courses: 0,
    enrollments: 0,
    attendance: 0
  });

  const loadDashboard =
    async () => {
      try {
        const [
          studentsResponse,
          coursesResponse,
          enrollmentResponse,
          attendanceResponse
        ] =
          await Promise.all([
            api.get(
              "/students?limit=1"
            ),

            api.get(
              "/courses"
            ),

            api.get(
              "/enrollments"
            ),

            api.get(
              `/attendance?date=${
                new Date()
                  .toISOString()
                  .split("T")[0]
              }`
            )
          ])
        setStats({
          students:
            studentsResponse
              .data
              .pagination

```

```

        .total,

        courses:
        coursesResponse
        .data
        .courses
        .length,

        enrollments:
        enrollmentResponse
        .data
        .enrollments
        .length,

        attendance:
        attendanceResponse
        .data
        .attendance
        .length
    });
} catch (
error
) {
    console.error(
        error
    );
}
};

useEffect(
    () => {
        loadDashboard();
    },
    []
);

const cards = [
    {
        title:
            "Total Students",
        value:
            stats.students
    },
    {
        title:
            "Total Courses",
        value:
            stats.courses
    },
    {
        title:
            "Total Enrollments",
        value:
            stats.enrollments
    },

```

```

    {
      title:
        "Today's Attendance",
      value:
        stats.attendance
    }
  ];

  return (
    <div>

      <h1 className="mb-6 text-3xl font-bold">
        Dashboard
      </h1>

      <div className="grid gap-6 sm:grid-cols-2 lg:grid-cols-4">

        {cards.map(
          (card) => (
            <div
              key={
                card.title
              }
              className="rounded-xl bg-white p-6 shadow"
            >
              <p className="text-sm text-gray-500">
                {
                  card.title
                }
              </p>

              <p className="mt-2 text-3xl font-bold">
                {
                  card.value
                }
              </p>
            </div>
          )
        )}

      </div>

    </div>
  );
};

export default Dashboard;

```

39. Students Page

Create the Students CRUD interface.

Required features:

- List students
- Search students
- Add student
- Edit student
- Delete student
- Student ID
- First name

-
- Last name
 - Email
 - Phone
 - Gender
 - Date of birth
 - Address
 - Guardian name
 - Guardian phone
 - Status
 - Loading state
 - Empty state
 - Error state
 - Delete confirmation

Use:

```
GET    /api/students
POST   /api/students
GET    /api/students/:id
PUT    /api/students/:id
DELETE /api/students/:id
```

The Students page should use Axios with `withCredentials: true`.

40. Add Student Page

Create a professional form with:

```
Student ID
First Name
Last Name
Email
Phone
Gender
Date of Birth
Address
Guardian Name
Guardian Phone
Status
```

Submit:

```
POST /api/students
```

After successful creation:

```
Navigate → /students
```

41. Edit Student Page

Load:

```
GET /api/students/:id
```

Allow editing all student fields.

Submit:

```
PUT /api/students/:id
```

After successful update:

```
Navigate → /students
```

42. Courses Page

Create complete Course CRUD.

Fields:

```
Course Code
Course Name
Description
Duration
Fee
Status
```

APIs:

```
GET    /api/courses
POST   /api/courses
GET    /api/courses/:id
PUT    /api/courses/:id
DELETE /api/courses/:id
```

Include:

- Search
 - Status filter
 - Add
 - Edit
 - Delete
 - Confirmation
 - Loading
 - Empty state
-

43. Add Course

Create:

```
frontend/src/pages/AddCourse.jsx
```

Form:

```
Course Code
Course Name
Description
Duration
Fee
Status
```

Submit:

```
POST /api/courses
```

44. Edit Course

Create:

```
frontend/src/pages/EditCourse.jsx
```

Load:

```
GET /api/courses/:id
```

Update:

```
PUT /api/courses/:id
```

45. Enrollment Page

Create Enrollment management.

Fields:

```
Student
Course
Enrollment Date
Batch
Status
```

Use populated data.

Example response:

```
{
  "student": {
    "studentId": "STU001",
    "firstName": "John",
    "lastName": "Perera"
  },
  "course": {
    "courseCode": "MERN001",
    "courseName": "MERN Stack Development"
  },
  "batch": "B001",
  "status": "Active"
}
```

APIs:

```
GET    /api/enrollments
POST   /api/enrollments
GET    /api/enrollments/:id
PUT    /api/enrollments/:id
DELETE /api/enrollments/:id
```

46. Add Enrollment

Load:

```
GET /api/students?limit=100
```

and:

```
GET /api/courses?status=Active
```

Create dropdowns:

```
Select Student
Select Course
```

Also include:

```
Enrollment Date
Batch
Status
```

Submit:

```
POST /api/enrollments
```

47. Attendance Page

Create Attendance management.

Fields:

```
Student
Course
Date
Status
Remarks
```

Status:

```
Present
Absent
Late
```

APIs:

```
GET    /api/attendance
POST   /api/attendance
GET    /api/attendance/:id
PUT    /api/attendance/:id
DELETE /api/attendance/:id
```

Support filtering by:

```
Date
Course
Student
Status
```

48. Mark Attendance Page

Create a professional attendance marking screen.

First select:

```
Course
Date
```

Then load active enrollments:

```
GET /api/enrollments?course=COURSE_ID&status=Active
```

Display all students in that course.

For every student show:

```
Student ID
Student Name
Present
Absent
Late
```

Default:

Present

When Save Attendance is clicked, create attendance records.

Example:

```
{
  "student": "STUDENT_ID",
  "course": "COURSE_ID",
  "date": "2026-09-24",
  "status": "Present",
  "remarks": ""
}
```

Do not allow duplicate attendance for the same:

Student + Course + Date

49. Profile Page

Display logged-in user:

Name
Email
Role

Use:

GET /api/auth/me

Do not expose the password.

50. App Routing

Configure these routes:

```
/login
/register

/dashboard

/students
/students/add
/students/:id/edit

/courses
/courses/add
/courses/:id/edit

/enrollments
/enrollments/add

/attendance
/attendance/mark

/profile
```

Protected routes must require authentication.

Unauthenticated users must be redirected to:

```
/login
```

51. Main App Layout

Use:

```
Navbar
+
Sidebar
+
Page Content
```

Desktop:

```
-----
|  Navbar  |
|-----|
| Sidebar | Main Content |
|         |               |
|         |               |
|         |               |
|-----|
```

Make the interface responsive.

On mobile, hide the desktop sidebar and provide a mobile navigation/menu.

52. UI Requirements

Use Tailwind CSS.

Design should be:

- Professional
- Clean
- Responsive
- Modern
- Dashboard-style
- Accessible
- Consistent

Use:

```
Cards
Tables
Forms
Badges
Buttons
Modals
Dropdowns
Search boxes
Pagination
Loading indicators
Empty states
Error messages
Success messages
```

Use Lucide React icons where appropriate.

53. Authentication Requirements

The authentication flow MUST be:

```
React
↓
POST /api/auth/login
↓
Express
↓
Validate credentials
↓
bcrypt password comparison
↓
Generate JWT
↓
Set JWT in httpOnly cookie
↓
Browser stores cookie
```

For subsequent requests:

```
React
↓
Axios request
↓
withCredentials: true
↓
Browser automatically sends cookie
↓
Express cookie-parser
↓
req.cookies.token
↓
jwt.verify()
↓
req.user
↓
Protected controller
```

The JWT must NEVER be accessible from React JavaScript.

Do NOT use:

```
localStorage
```

for authentication.

Do NOT use:

```
sessionStorage
```

for authentication.

Do NOT expose the JWT in JSON responses.

54. Cookie Configuration

Development:

```
{
  httpOnly: true,
  secure: false,
  sameSite: "lax"
}
```

Production:

```
{
  httpOnly: true,
  secure: true,
  sameSite: "none"
}
```

When frontend and backend are deployed on different sites/domains, configure CORS:

```
app.use(  
  cors({  
    origin: process.env.CLIENT_URL,  
    credentials: true  
  })  
);
```

Never use:

```
origin: ""
```

with credentialed cookies.

55. API List

Authentication

```
POST /api/auth/register  
POST /api/auth/login  
POST /api/auth/logout  
GET /api/auth/me
```

Students

```
GET /api/students  
POST /api/students  
GET /api/students/:id  
PUT /api/students/:id  
DELETE /api/students/:id
```

Courses

```
GET /api/courses  
POST /api/courses  
GET /api/courses/:id  
PUT /api/courses/:id  
DELETE /api/courses/:id
```

Enrollments

```
GET /api/enrollments  
POST /api/enrollments  
GET /api/enrollments/:id  
PUT /api/enrollments/:id  
DELETE /api/enrollments/:id
```

Attendance

```
GET    /api/attendance
POST   /api/attendance
GET    /api/attendance/:id
PUT    /api/attendance/:id
DELETE /api/attendance/:id
```

56. Postman Testing

Test in this order.

Step 1

Register:

```
POST http://localhost:5000/api/auth/register
```

Body:

```
{
  "name": "Admin User",
  "email": "admin@example.com",
  "password": "123456"
}
```

Step 2

Login:

```
POST http://localhost:5000/api/auth/login
```

Body:

```
{
  "email": "admin@example.com",
  "password": "123456"
}
```

The JWT must be stored in the cookie.

Step 3

Test:

```
GET http://localhost:5000/api/auth/me
```

Step 4

Create Student.

```
POST http://localhost:5000/api/students
```

Body:

```
{
  "studentId": "STU001",
  "firstName": "John",
  "lastName": "Perera",
  "email": "john@example.com",
  "phone": "0771234567",
  "gender": "Male",
  "dateOfBirth": "2002-05-15",
  "address": "Kandy",
  "guardianName": "Mr. Perera",
  "guardianPhone": "0777654321",
  "status": "Active"
}
```

Step 5

Create Course.

```
POST http://localhost:5000/api/courses
```

Body:

```
{
  "courseCode": "MERN001",
  "courseName": "MERN Stack Development",
  "description": "Full Stack Web Development",
  "duration": "6 Months",
  "fee": 75000,
  "status": "Active"
}
```

Step 6

Create Enrollment.

```
POST http://localhost:5000/api/enrollments
```

Body:

```
{
  "student": "STUDENT_OBJECT_ID",
  "course": "COURSE_OBJECT_ID",
  "enrollmentDate": "2026-09-24",
  "batch": "B001",
  "status": "Active"
}
```

Step 7

Create Attendance.

```
POST http://localhost:5000/api/attendance
```

Body:

```
{
  "student": "STUDENT_OBJECT_ID",
  "course": "COURSE_OBJECT_ID",
  "date": "2026-09-24",
  "status": "Present",
  "remarks": ""
}
```

57. Development Order

Build the application in this order:

Phase 1
Backend project setup

Phase 2
MongoDB connection

Phase 3
Mongoose models

Phase 4
Authentication

Phase 5
JWT httpOnly cookie

Phase 6
Authentication middleware

Phase 7
Student CRUD

Phase 8
Course CRUD

Phase 9
Enrollment CRUD

Phase 10
Attendance CRUD

Phase 11
Postman testing

Phase 12
React project setup

Phase 13
Axios + credentials

Phase 14
Auth Context

Phase 15
Protected Routes

Phase 16
Login/Register

Phase 17
Dashboard

Phase 18
Students UI

Phase 19

Courses UI

Phase 20
Enrollment UI

Phase 21
Attendance UI

Phase 22
Profile

Phase 23
Responsive design

Phase 24
Validation and error handling

Phase 25
Final testing

58. Important Error Handling

The application must properly handle:

Invalid login
Invalid JWT
Expired JWT
Missing cookie
Duplicate email
Duplicate student ID
Duplicate course code
Duplicate enrollment
Duplicate attendance
Invalid MongoDB ID
Required fields
Invalid form values
Server errors
Network errors

Display user-friendly messages.

Do not expose stack traces to frontend users.

59. Final Feature Checklist

Authentication:

[] Register
[] Login
[] Logout
[] JWT
[] httpOnly Cookie
[] Protected Routes
[] Auth Context
[] Current User

Students:

```
[ ] Create
[ ] Read
[ ] Update
[ ] Delete
[ ] Search
[ ] Status Filter
[ ] Pagination
```

Courses:

```
[ ] Create
[ ] Read
[ ] Update
[ ] Delete
[ ] Search
[ ] Status
```

Enrollments:

```
[ ] Create
[ ] Read
[ ] Update
[ ] Delete
[ ] Student relationship
[ ] Course relationship
[ ] populate()
```

Attendance:

```
[ ] Mark attendance
[ ] Present
[ ] Absent
[ ] Late
[ ] Date filter
[ ] Course filter
[ ] Student filter
[ ] Duplicate prevention
```

Dashboard:

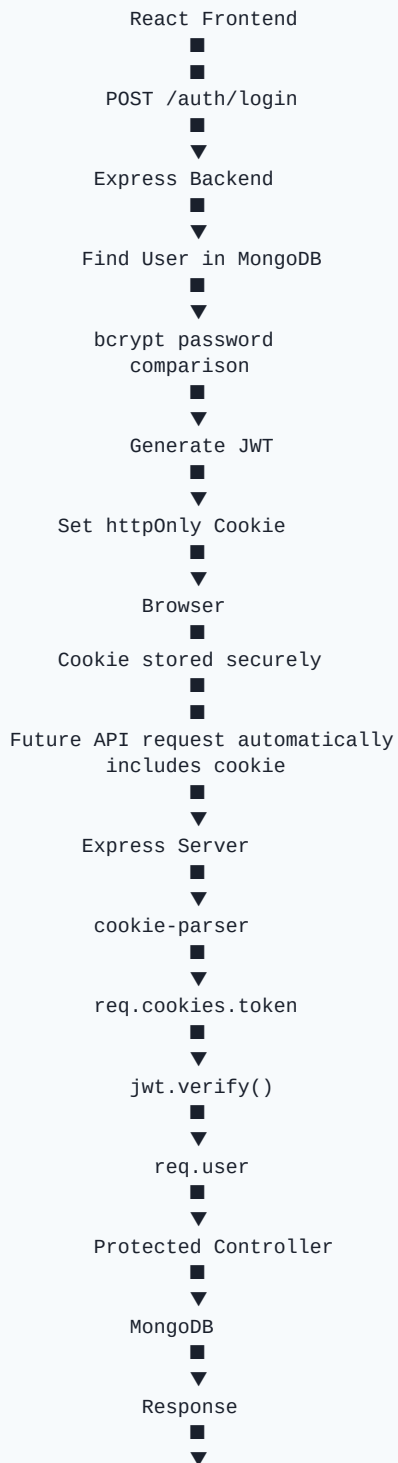
```
[ ] Total students
[ ] Total courses
[ ] Total enrollments
[ ] Today's attendance
```

Frontend:

```
[ ] React
[ ] Vite
[ ] Tailwind
[ ] React Router
[ ] Axios
[ ] Lucide icons
[ ] Responsive layout
[ ] Loading states
[ ] Empty states
[ ] Error states
[ ] Confirmation dialogs
```

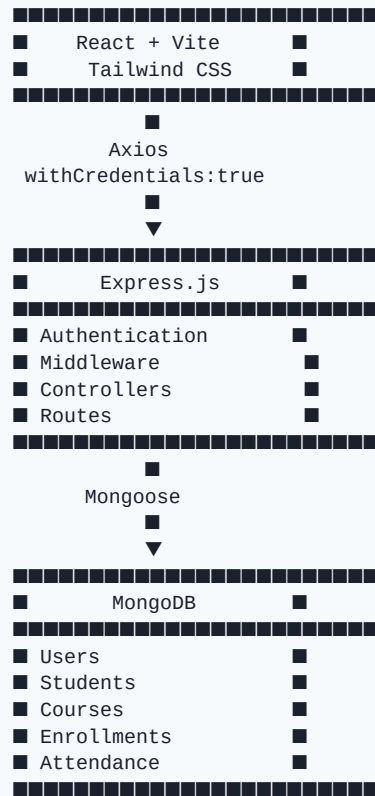
60. Final Authentication Architecture

The final authentication system must look like this:



React Frontend

61. Final Project Architecture



The completed application should function as a real CRUD-based Student Management System with authentication, database relationships, enrollment management, attendance management, dashboard statistics, responsive UI, and secure JWT authentication using **httpOnly** cookies.

Do not skip files, routes, controllers, models, or authentication logic. Implement the project incrementally and ensure every frontend API call matches the backend API exactly.